

Stackademic

Integrating Firebase Cloud Messaging for Real-time Push Notifications in Android



Diego Marcher

Follow

3 min read · Sep 13, 2023



103





Push notifications are a critical component of modern mobile applications, enhancing user engagement and providing real-time updates. Firebase Cloud Messaging (FCM) is a robust platform by Google that enables

developers to send notifications to Android devices. In this article, we'll explore how to integrate FCM into an Android application for instant chat messaging using Kotlin. We'll also discuss unit testing and best practices to ensure a smooth notification experience.

Medium Search

Get app

 Write

Sign up

Sign in



Prerequisites

Before we dive into the implementation, make sure you have the following prerequisites in place:

1. Android Studio with a sample Android project.
2. A Firebase project created on the Firebase Console (<https://console.firebase.google.com>).
3. Firebase Cloud Messaging added to your Firebase project.

Setting up Firebase Cloud Messaging

1. Add Firebase to your Android project

In your Android Studio project, navigate to `Tools > Firebase` and select "Cloud Messaging." Follow the setup wizard to connect your app to Firebase.

2. Configure the `AndroidManifest.xml`

Firebase requires configuration settings in your `AndroidManifest.xml` . Add the following within the `<application>` tag:

```
<!-- Firebase Cloud Messaging -->
<service
    android:name=".MyFirebaseMessagingService">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

3. Create a Firebase Messaging Service

Create a class named `MyFirebaseMessagingService` that extends `FirebaseMessagingService` . This service handles incoming messages and displays notifications to the user:

```
class MyFirebaseMessagingService : FirebaseMessagingService() {
    override fun onMessageReceived(remoteMessage: RemoteMessage) {
        // Handle incoming message and display notification.
    }
}
```

4. Send test notifications:

Use the Firebase Console to send a test notification to your app. This helps verify that your setup is correct.

Implementing Instant Chat Messaging

Now, let's extend our chat application to handle real-time messaging using FCM. We'll focus on sending and receiving chat messages.

1. Sending Chat Messages

When a user sends a chat message, use FCM to send the message content to the recipient's device. You can include sender and message information in the FCM payload.

```
fun sendChatMessage(recipientToken: String, messageContent: String) {  
    // Construct the FCM message payload.  
    val payload = mapOf(  
        "sender" to "John",  
        "message" to messageContent  
    )  
    // Send the FCM message to the recipient's device.  
    FirebaseMessaging.getInstance().send(  
        RemoteMessage.Builder(recipientToken)  
            .setMessageType("chat")  
            .setData(payload)  
            .build()  
    )  
}
```

```
)  
}
```

2. Receiving Chat Messages

In your `MyFirebaseMessagingService`, handle incoming chat messages and display them in your chat interface.

```
override fun onMessageReceived(remoteMessage: RemoteMessage) {  
    with(message.data) {  
        val sender = get("sender") ?: return  
        val message = get("message") ?: return  
        displayChatMessage(sender, message)  
    }  
}
```

Unit Testing

Testing is a crucial part of any application. To unit test your FCM implementation, you can use a testing framework like Mockito to mock Firebase services. Ensure that your message handling logic works correctly by writing unit tests for `MyFirebaseMessagingService`.

Get Diego Marcher's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Here's an example of a unit test using Mockito and JUnit:

```
@RunWith(MockitoJUnitRunner::class)
class MyFirebaseMessagingServiceTest {

    @InjectMocks
    private lateinit var messagingService: MyFirebaseMessagingService

    @Mock
    private lateinit var remoteMessage: RemoteMessage

    @Test
    fun testOnMessageReceived() {
        // Mock message data
        `when`(remoteMessage.data).thenReturn(createMessageData())
        // Call the method to test
        messagingService.onMessageReceived(remoteMessage)
        // Assert the expected behavior
        // ...
    }

    private fun createMessageData(): Map<String, String> {
```

```
val data = mutableMapOf(  
    "sender" to "Alice",  
    "message" to "Hello, there!"  
)  
return data  
}
```

That's it!

Happy coding!

Stackademic

Thank you for reading until the end. Before you go:

- Please consider **clapping** and **following** the writer! 🙌
- Follow us on [Twitter\(X\)](#), [LinkedIn](#), and [YouTube](#).
- Visit [Stackademic.com](#) to find out more about how we are democratizing free programming education around the world.

[Android](#)[Kotlin](#)[Firebase](#)[Notifications](#)[Software Development](#)



Published in Stackademic

85K followers · Last published 3 hours ago

Follow

Stackademic is a learning hub for programmers, devs, coders, and engineers. Our goal is to democratize free coding education for the world.



Written by Diego Marcher

409 followers · 9 following

Follow

Senior Android Engineer

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Store](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)